Module 9 Week A — Reading: Knowledge Graphs, RDF & SPARQL

Everything in this reading is fair game for readiness checks, drills, and labs. A learner who reads this carefully should be able to complete the drill and most of the lab without attending the concept session.

What success looks like: By the end of Week A, you can model a small domain as an RDF graph using subject–predicate–object triples with consistent URI conventions, serialize it as Turtle, stand up an Apache Jena Fuseki triple store via Docker and load a TTL dataset, write SPARQL SELECT/CONSTRUCT/ASK queries with WHERE/FILTER/OPTIONAL/ORDER BY/LIMIT clauses, apply RDFS/OWL/SKOS vocabularies for disambiguation, and use both `rdflib` and `SPARQLWrapper` from Python.

In This Reading

1. Setup: install before the concept session
2. Why a knowledge graph?
3. The RDF triple model
4. Namespaces, prefixes, and Turtle serialization
5. Apache Jena Fuseki — the triple store
6. SPARQL — the query language
7. Ontology vocabularies (RDFS, OWL, SKOS) and disambiguation
8. The Python toolkit — rdflib and SPARQLWrapper

- Troubleshooting
- Key Terms
- Summary
- What's Next

---

1. Setup: install before the concept session

Run these commands before Sunday's concept session. Most of this is fast; the only step that needs real wall-clock is the Fuseki Docker image pull (~3 min on first run, ~150 MB).

1.1 Python packages. From your course virtualenv:

```
pip install -r requirements.txt
```

For Module 9 Week A, `requirements.txt` adds two packages:

```
rdflib>=6.0,<7
SPARQLWrapper>=2.0,<3
```

Both are pure-Python wheels. Install in seconds.

1.2 Fuseki Docker image. Pull the image once now so the lab autograder isn't waiting on a download:

```
docker pull stain/jena-fuseki:5.0.0
```

Expect ~3 minutes on first run; subsequent runs are cached.

Platform notes:

- **macOS / Linux:** Docker Desktop or Docker Engine must already be running before `docker pull`. On Linux without Desktop, ensure your user is in the `docker` group (`sudo usermod -aG docker $USER`) or prefix commands with `sudo`.
- **Windows:** Docker Desktop must be running and WSL 2 must be enabled. If `docker pull` fails with `error during connect`, open Docker Desktop and wait for the whale icon to settle before retrying.

1.3 Verify. With Docker Desktop running, do a one-off smoke test of Fuseki — this catches port conflicts and bring-up issues before Monday's lab:

```
docker run --rm -p 3030:3030 stain/jena-fuseki:5.0.0
```

In another terminal:

```
curl -s http://localhost:3030/$/ping
```

You should see a single timestamp line. Stop the container with `Ctrl-C` in the first terminal. If port 3030 is already in use on your machine, pick another free port (`-p 3031:3030`) and remember that change for the lab — you will need to point `docker-compose.yml` at the alternate host port.

1.4 What carries through. The same `pip install` and Docker pull also serve the lab and integration repos via pip's cache (subsequent installs are ~30 seconds) and Docker's image cache (no re-pull). The drill is the exception: it uses `rdflib` in-memory and never touches Fuseki — you do not need Docker running to do the drill.

---

2. Why a knowledge graph?

You already have one retrieval substrate from Module 8: a vector database. You hand it a query string, it returns passages similar to that string in embedding space. That works beautifully for "find me text that says something close to this" — which is most of what production RAG systems do.

It works poorly for a different shape of question:

> *"Which 2018 papers were cited by both Bengio and Hinton?"*

Try writing that as a vector query. You can't — there isn't a passage that *means* this. The question is about **structured relationships** (cited-by, author, year), not similarity. The right substrate for it is a graph where authors, papers, venues, years, and citation links are first-class nodes and edges. That is a **knowledge graph (KG)**.

A knowledge graph is a directed, labeled graph. Each node is a thing (an author, a paper, a topic). Each edge is a typed relationship between things (`:authored`, `:cites`, `:publishedIn`). Once your data is in this shape, multi-hop questions become *trivial* and similarity questions become *awkward*. The trade is the opposite of the vector database.

A useful frame:

Question shape	Best substrate
"Find me passages similar to *X*"	Vector retrieval
"Find me papers cited by both *A* and *B*"	Knowledge graph
"What does this paragraph say about *X*?"	Vector retrieval → generator (RAG)
"List all of *A*'s coauthors at *Venue V* in 2023"	Knowledge graph (SPARQL)
"Summarize the structured facts about *Entity E*"	Both — KG for facts, vector for sources

Most production systems combine both: vector retrieval for fuzzy text recall, KG lookups for structured facts. This is the point Week B builds toward. Week A focuses on the KG side in isolation, so you have the substrate ready when integration time comes.

Three additional reasons knowledge graphs earn their keep in real systems:

1. **Schema flexibility.** Adding a new property to one node does not require an `ALTER TABLE` migration. The graph absorbs heterogeneous data — recipes from one source, books from another, authors from a third — and they all live side by side without coordination.
2. **Identity resolution across sources.** Two records about the same author from different sources merge into the same node by URI identity. No join keys to design.
3. **Traversal queries.** Graph queries are pattern-matching over the shape of relationships, not row scans. "Show me everyone three steps from *X*" is a normal query, not a recursive CTE workaround.

---

3. The RDF triple model

RDF stands for Resource Description Framework. It is the W3C standard for representing facts as graphs, and it is the data model every triple store, including Fuseki, implements.

Every fact in RDF is a **triple**: three terms in a fixed order.

```
subject  predicate  object
```

Read each triple left to right as a sentence:

```
:alice    :worksAt   :acme            # "Alice works at Acme."
:acme     :foundedIn  1995            # "Acme was founded in 1995."
:alice    :knows     :bob             # "Alice knows Bob."
```

The subject is always an entity (an IRI — see below). **The predicate** is always a relation (also an IRI). **The object** is either another entity (an IRI) or a **literal** value (a number, a string, a date).

A graph is just a set of triples. There is no enclosing object, no table, no schema. The same subject can appear in any number of triples; you describe it as completely or as sparsely as you like. Two triples with the same subject describe the same entity — that is how a graph emerges from a flat list.

Subjects and objects: IRIs, literals, blank nodes

Three kinds of terms can fill the subject or object slot:

1. **IRI** (Internationalized Resource Identifier — the same idea as a URL or URI, generalized to non-ASCII characters). IRIs identify entities globally. They look like `http://example.org/people/alice` and they are written between angle brackets: `<http://example.org/people/alice>`. Predicates are always IRIs.
2. **Literal** — a plain value with a datatype: `"Alice"`, `42`, `2026-05-28`. Literals can only fill the object slot, never the subject or predicate.
3. **Blank node** — an anonymous entity. Useful when you have structure that has no natural identifier (e.g., a temporary container). Written as `_:b1`. You will see blank nodes occasionally; you do not author them often in Week A.

The point of IRIs is **global identifiers**. Two datasets that both use `<http://example.org/people/alice>` agree they are talking about the same Alice. That is how you merge data from multiple sources without a join key.

Predicates: typed edges

The predicate gives the edge a **type**. `:worksAt` is a different edge type from `:knows`. A KG query like "find all of Alice's colleagues" walks specifically the `:worksAt` edges, not arbitrary ones. This is what makes graphs *typed* — your queries can filter on edge kind.

What is *not* in RDF

There is no table, no schema, no primary key, no foreign key constraint, no `NULL`. Missing facts are just absent triples. If `:alice :worksAt :acme` is in the graph but `:alice :birthYear ...` is not, Alice simply has no recorded birth year. A `SELECT` that asks for both joins on what exists.

This is liberating until you realize it is also the source of every "my query returned 0 rows" surprise — the data might not be wrong; the triple you assumed exists might not.

---

4. Namespaces, prefixes, and Turtle serialization

Writing `<http://example.org/recipes/lasagna>` ten times in a file is unworkable. RDF inherits the XML solution: **namespace prefixes**.

A **namespace** is the leading common part of a set of IRIs. A **prefix** is a short alias. After declaring the prefix, you write the short form in place of the full IRI.

Turtle (TTL) — the human-readable serialization

RDF has several serializations (N-Triples, JSON-LD, RDF/XML). The one you will read and write 95% of the time is **Turtle**, file extension `.ttl`. It is the most legible and is what Fuseki accepts by default.

A minimal Turtle file:

```
@prefix : <http://example.org/recipes/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

:lasagna  rdfs:label   "Lasagna" ;
          :cuisine     :Italian ;
          :year        2019 .

:Italian  rdfs:label   "Italian cuisine" .
```

Three things to notice:

1. **`@prefix` declarations** at the top. After `@prefix : <http://example.org/recipes/> .`, the bare colon `:lasagna` expands to `<http://example.org/recipes/lasagna>`.
2. **Semicolons let you share a subject.** `:lasagna rdfs:label "Lasagna" ; :cuisine :Italian ; :year 2019 .` is three triples with the same subject. The `;` continues with a new predicate–object pair. The `.` terminates the subject group.
3. **Literals.** `"Lasagna"` is a string literal. `2019` is an integer literal (Turtle infers the datatype). Dates take the form `"2026-05-28"^^xsd:date`.

Commas chain multiple objects on the same predicate:

```
:lasagna :author :emma , :liam .
```

That is two triples: `:lasagna :author :emma .` and `:lasagna :author :liam .`.

Common predeclared vocabularies

You will see these prefixes constantly:

```
@prefix rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
```

Each is a well-known **vocabulary** — a fixed set of IRIs with agreed meanings. We will reach for `rdfs`, `owl`, and `skos` in §7.

`a` is sugar for `rdf:type`

A line you will see all the time:

```
:lasagna a :Recipe .
```

This is a triple `:lasagna rdf:type :Recipe .` — "Lasagna is an instance of the class `:Recipe`." Turtle abbreviates `rdf:type` to the keyword `a` for readability.

---

5. Apache Jena Fuseki — the triple store

Once you have a Turtle file, you need somewhere to **persist** and **query** it. That is a **triple store**: a database whose primary unit of storage is the triple and whose query interface is SPARQL over HTTP.

We use **Apache Jena Fuseki**, the reference implementation from the Apache Jena project. The course's chosen Docker image is `stain/jena-fuseki:5.0.0` — the actively maintained community build pinned to a specific version.

What Fuseki gives you

- A persistent triple store (TDB2 backend, written to disk).
- One or more **datasets** — each a named, isolated graph. The lab dataset will be `/recipes`; the integration dataset will be `/publications`.
- An HTTP **SPARQL endpoint** per dataset. `POST` queries to `http://localhost:3030/recipes/sparql` and `POST` data to `http://localhost:3030/recipes/data`.
- A web UI at `http://localhost:3030` for ad-hoc queries and dataset administration.

Standing it up

The lab and integration both come with a `docker-compose.yml`. The minimal form:

```
services:
  fuseki:
    image: stain/jena-fuseki:5.0.0
    ports:
      - "3030:3030"
    environment:
      ADMIN_PASSWORD: admin
      FUSEKI_DATASET_1: recipes
    volumes:
      - fuseki-data:/fuseki

volumes:
  fuseki-data:
```

Then:

```
docker compose up -d
```

Fuseki is ready when `curl -s http://localhost:3030/$/ping` returns a timestamp.

Loading data

Two paths to load a TTL file into a dataset called `recipes`:

Via HTTP (what the lab autograder uses):

```
curl -X POST \
  -u admin:admin \
  -H "Content-Type: text/turtle" \
  --data-binary @recipes.ttl \
  http://localhost:3030/recipes/data
```

Via Python (`load_dataset.py` in the lab starter does this):

```
import requests
with open("recipes.ttl", "rb") as f:
    r = requests.post(
        "http://localhost:3030/recipes/data",
        headers={"Content-Type": "text/turtle"},
        auth=("admin", "admin"),
        data=f.read(),
    )
r.raise_for_status()
```

The `-u admin:admin` / `auth=(...)` is required because `docker-compose.yml` starts Fuseki with `ADMIN_PASSWORD: admin`, which gates write endpoints (POST `/recipes/data`, POST `/recipes/update`) behind HTTP Basic Auth. The read endpoint (`/recipes/sparql` for SELECT/ASK/CONSTRUCT queries) is open — no auth needed.

After loading, you can verify in two seconds with `ASK { ?s ?p ?o }`:

```
curl -G http://localhost:3030/recipes/sparql \
  --data-urlencode "query=ASK { ?s ?p ?o }" \
  -H "Accept: application/json"
```

You should see `{"head":{},"boolean":true}`.

---

6. SPARQL — the query language

SPARQL (pronounced "sparkle") is to RDF what SQL is to relational tables. It is a W3C standard with four query forms; you will use three of them in Week A.

The three forms you will use

Form	Returns	Use it when
`SELECT`	A table of rows (variable bindings)	You want columns of values
`CONSTRUCT`	A new graph (set of triples)	You want to reshape or extract a subgraph
`ASK`	A single boolean	You want a yes/no

(The fourth form, `DESCRIBE`, returns "everything about" a resource. You will not need it in Week A.)

Anatomy of a query

```
PREFIX :     <http://example.org/recipes/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?recipe ?label
WHERE {
  ?recipe a :Recipe ;
          rdfs:label ?label .
}
ORDER BY ?label
LIMIT 5
```

Three structural pieces:

1. **PREFIX block.** Same shape as Turtle's `@prefix`, minus the `@` and trailing `.`. Declares the abbreviations you will use in the rest of the query.
2. **Query form clause.** `SELECT ?recipe ?label` says "return two columns named `recipe` and `label`." Variables start with `?`.
3. **WHERE block.** A **basic graph pattern (BGP)** — triples to match, with variables in some slots. The engine searches the graph for all subgraphs that fit the pattern, and binds the variables to whatever fills them.

`ORDER BY` and `LIMIT` work just like in SQL.

How matching works

The WHERE block is read as **"find every set of bindings for the variables such that all triples in the block exist in the graph."** Each matching binding is a row. There is no implicit `SELECT *`; you choose which bindings to project.

A triple in the WHERE block with one variable (e.g., `?recipe a :Recipe`) is a filter: keep only subjects that are recipes. A triple with two variables (e.g., `?recipe :author ?author`) is a join across the graph: match any recipe and its author.

`FILTER` — restrict bindings further

```
SELECT ?recipe ?year
WHERE {
  ?recipe a :Recipe ;
          :year ?year .
  FILTER (?year > 2020)
}
```

`FILTER` takes an expression and keeps only the bindings where it is true. It is the closest equivalent to a SQL `WHERE` predicate clause. Common operators: `<`, `>`, `=`, `!=`, `&&`, `||`, `regex()`, `str()`.

`OPTIONAL` — left-outer-join semantics

```
SELECT ?paper ?doi
WHERE {
  ?paper a :Paper .
  OPTIONAL { ?paper :doi ?doi }
}
```

This returns every paper. Where a paper has a `:doi` triple, `?doi` is bound. Where it does not, `?doi` is unbound and the row still appears. This is exactly SQL's `LEFT OUTER JOIN`.

A common bug: putting the optional triple in the main WHERE block instead of inside `OPTIONAL { ... }`. Papers without a DOI silently drop from the results. We will revisit this in Troubleshooting.

`CONSTRUCT` — return a graph

```
CONSTRUCT {
  ?recipe :displayName ?name .
}
WHERE {
  ?recipe a :Recipe ;
          rdfs:label ?name .
}
```

`CONSTRUCT` says: "for every match in the WHERE block, emit the triples in the template (the first `{ ... }` block)." The result is a graph — useful for ETL (Extract, Transform, Load), dataset reshaping, or extracting a summary subgraph.

`ASK` — boolean

```
ASK {
  ?recipe :cuisine :Italian .
}
```

Returns `true` if at least one match exists; otherwise `false`. Cheap: the engine can stop on the first hit.

---

7. Ontology vocabularies (RDFS, OWL, SKOS) and disambiguation

So far the predicates you have seen — `:recipe`, `:cuisine`, `:author` — are project-defined. RDF has three standard **vocabularies** of pre-agreed predicates that every triple store understands and that solve specific recurring problems. You should reach for them rather than invent your own.

RDFS — types and labels

RDFS (RDF Schema) gives you the apparatus of classes and labels.

Predicate	Meaning
`rdf:type` (often written `a`)	"this resource is an instance of this class"
`rdfs:Class`	a class — a kind of thing
`rdfs:label`	a human-readable name for the resource
`rdfs:subClassOf`	"every instance of class A is also an instance of class B"

Example:

```
:Italian rdfs:subClassOf :European .
:lasagna a :Italian .
```

A reasoning-aware query for `:European` recipes will return `:lasagna` because the engine knows Italian ⊆ European. (Plain Fuseki does not do this reasoning by default — you have to enable RDFS entailment on the dataset or ask the query for the subclass walk explicitly. We will not turn on entailment in Week A; that comes in Week B.)

OWL — richer class semantics

OWL (Web Ontology Language) extends RDFS with constructs like `owl:sameAs`, `owl:equivalentClass`, and many more. The one you will see most often in Week A is `owl:sameAs`:

```
:nyc owl:sameAs :newYorkCity .
```

Asserts that `:nyc` and `:newYorkCity` are the same entity — useful when two sources used different IRIs for the same thing.

SKOS — controlled vocabularies and labels

SKOS (Simple Knowledge Organization System) is the vocabulary for **controlled vocabularies and disambiguation**.

Predicate	Meaning
`skos:prefLabel`	the canonical, preferred string label for this concept
`skos:altLabel`	an alternative string that also refers to this concept
`skos:broader`	this concept is narrower than another
`skos:narrower`	this concept is broader than another

The disambiguation pattern:

```
:Italian
  skos:prefLabel "Italian" ;
  skos:altLabel "italiano" , "ital." , "ITA" .
```

A query that wants to match any of those surface forms checks **both** `skos:prefLabel` and `skos:altLabel`:

```
SELECT ?cuisine
WHERE {
  ?cuisine ?label "italiano" .
  FILTER (?label = skos:prefLabel || ?label = skos:altLabel)
}
```

This is the bridge between text-shaped questions ("any recipe with `italiano` as its cuisine label") and the structured graph (a single canonical `:Italian` URI).

Ontology-based disambiguation in one sentence

When your data has surface-form variants for the same underlying concept, give each concept a single canonical URI plus `skos:prefLabel` and any number of `skos:altLabel` s, and write queries that look up either label. Week B does exactly this against NER (Named Entity Recognition) outputs.

---

8. The Python toolkit — rdflib and SPARQLWrapper

You have two ways to run SPARQL from Python. Use the right one for the job.

`rdflib` — in-memory graphs and queries

`rdflib` parses Turtle into a Python object you can query directly, with no triple store involved.

```
from rdflib import Graph

g = Graph()
g.parse("recipes.ttl", format="turtle")

query = """
PREFIX : <http://example.org/recipes/>
SELECT ?r ?label WHERE { ?r a :Recipe ; rdfs:label ?label }
"""
for row in g.query(query):
    print(row.r, row.label)
```

`row.r` is an `rdflib.term.URIRef`; `row.label` is an `rdflib.term.Literal`. Both behave like strings most of the time. Call `.toPython()` if you need a plain Python `str` or `int`.

Use `rdflib` when: your data is small (thousands to low millions of triples), fits in memory, lives in a `.ttl` file, and you want zero infrastructure. The drill uses `rdflib` exclusively.

`SPARQLWrapper` — remote endpoint client

`SPARQLWrapper` sends queries over HTTP to a SPARQL endpoint like Fuseki.

```
from SPARQLWrapper import SPARQLWrapper, JSON

sparql = SPARQLWrapper("http://localhost:3030/recipes/sparql")
sparql.setReturnFormat(JSON)
sparql.setQuery("""
PREFIX : <http://example.org/recipes/>
SELECT ?r WHERE { ?r a :Recipe }
""")
results = sparql.query().convert()
for binding in results["results"]["bindings"]:
    print(binding["r"]["value"])
```

Two things easy to forget:

1. **Always call `setReturnFormat(JSON)`** — the default is XML, which is harder to parse.
2. **The result shape is nested JSON,** not a tidy row iterator. You walk `results["results"]["bindings"]` and each binding is a dict of variable → `{"type": ..., "value": ...}`.

Use `SPARQLWrapper` when: your data lives in Fuseki (or any remote endpoint), or you need to query a dataset too large to load into memory. The lab and integration use both `rdflib` (to validate the local TTL file before loading) and `SPARQLWrapper` (to query Fuseki after loading).

Troubleshooting

My Turtle file won't parse. The two most common culprits are (a) a missing `@prefix` declaration for a prefix used later — every prefix you reference must be declared in the file's header; and (b) an unterminated triple — every subject group ends with `.` and every predicate–object pair on a continued subject ends with `;` except the last, which ends with `.`. Run `python -c "from rdflib import Graph; Graph().parse('your_file.ttl', format='turtle')"` and the error message points at the line.

`docker compose up` fails: "port already allocated." Port 3030 is in use. Either stop the conflicting process or remap to a free host port (`"3031:3030"` in the compose file). Update any code or commands that hit `localhost:3030` accordingly.

Fuseki starts but the dataset isn't there. Check the `FUSEKI_DATASET_1` environment variable in `docker-compose.yml` matches the dataset name in your code. Visit `http://localhost:3030` and confirm the dataset appears in the UI. If you change the dataset name after the volume is created, delete the volume (`docker volume rm <name>`) so Fuseki re-initializes.

`POST` to `/recipes/data` returns 404. The dataset doesn't exist (see above). Or you posted to `/sparql` instead of `/data`. `/data` accepts triple uploads; `/sparql` accepts queries.

`POST` to `/recipes/data` returns 415. Wrong `Content-Type` header. Use `text/turtle` for TTL, `application/n-triples` for N-Triples, `application/ld+json` for JSON-LD.

`POST` to `/recipes/data` returns 401. You forgot HTTP Basic Auth. Fuseki's write endpoints are gated by `ADMIN_PASSWORD`. Add `-u admin:admin` (curl) or `auth=("admin", "admin")` (Python `requests`) to your POST. SELECT/ASK/CONSTRUCT queries to `/recipes/sparql` are unauthenticated — the auth only applies to writes.

My `SELECT` returns zero rows. This is almost never an engine bug. In order of likelihood: (1) a prefix declaration is missing or wrong — the query expanded to a different IRI than your data uses; (2) the predicate IRI in your query does not exactly match the one in your data (typos, casing); (3) the triples you assume exist actually don't — open the data in `rdflib` and inspect; (4) an `OPTIONAL` is missing where you expected one and your join dropped matching rows.

My `OPTIONAL` query returns no rows. `OPTIONAL { ?s :p ?o }` does not on its own match anything if the main WHERE block is empty — you need at least one mandatory pattern before the optional. Restructure: `?s a :Paper . OPTIONAL { ?s :p ?o }`.

`SPARQLWrapper` returns XML when I asked for JSON. You forgot `setReturnFormat(JSON)`. Add the line after constructing the wrapper.

`SPARQLWrapper` returns `URIRef` instead of a Python string. That is the right shape from `rdflib` 's side, not `SPARQLWrapper` 's. If you are using `rdflib.Graph.query`, call `.toPython()` on the result terms to get plain values. If you are using `SPARQLWrapper`, the result already is plain JSON — read `binding["x"]["value"]`.

My `ASK` returns nothing. `ASK` returns `True` or `False`, not rows. From `rdflib.Graph.query(...)`, treat the result as a single boolean: `bool(list(g.query(ask_q))[0])` or just iterate and inspect.

Key Terms

- **RDF** — Resource Description Framework. The W3C standard for graph-shaped data.
- **Triple** — a fact in RDF: subject–predicate–object.
- **IRI** — Internationalized Resource Identifier. A globally unique identifier for an entity, written `<http://example.org/...>`.
- **Literal** — a plain value (string, number, date) in the object slot of a triple.
- **Blank node** — an anonymous node, written `_:b1`. Rarely authored by hand in Week A.
- **Turtle (TTL)** — the human-readable RDF serialization.
- **Namespace** — a common IRI prefix for a set of related identifiers.
- **Prefix** — a short alias for a namespace (e.g., `skos:`).
- **`@prefix` (Turtle) / `PREFIX` (SPARQL)** — directives that declare the namespace alias.
- **Triple store** — a database whose unit of storage is the triple and whose query interface is SPARQL.
- **Apache Jena Fuseki** — the triple store used in this course.
- **Dataset** — a named, isolated graph inside a Fuseki instance (e.g., `/recipes`).
- **SPARQL endpoint** — the HTTP URL where you POST queries to a dataset (e.g., `http://localhost:3030/recipes/sparql`).
- **Basic graph pattern (BGP)** — the triples inside a `WHERE` block, against which the engine matches.
- **`SELECT` / `CONSTRUCT` / `ASK` / `DESCRIBE`** — the four SPARQL query forms. You use the first three in Week A.
- **`WHERE` / `FILTER` / `OPTIONAL` / `ORDER BY` / `LIMIT`** — the clauses that shape a query's binding set.
- **RDFS** — RDF Schema. Standard vocabulary for classes, instances, labels, and subclasses.
- **OWL** — Web Ontology Language. Standard vocabulary for richer class semantics; you will see `owl:sameAs` most often.
- **SKOS** — Simple Knowledge Organization System. Standard vocabulary for controlled vocabularies and surface-form disambiguation.
- **`skos:prefLabel`** — the canonical string label for a concept.
- **`skos:altLabel`** — an alternative string that also refers to a concept.
- **Ontology** — a vocabulary of classes, properties, and constraints that defines how a domain is modeled.
- **Ontology-based disambiguation** — resolving a surface form (e.g., `"italiano"`) to a canonical entity URI by looking up `skos:prefLabel` and `skos:altLabel`.
- **`rdflib`** — Python library for parsing TTL and running SPARQL against in-memory graphs.
- **`SPARQLWrapper`** — Python library for sending SPARQL queries to a remote endpoint over HTTP.

Summary

A knowledge graph is the right substrate for questions about **structured relationships** between entities — multi-hop traversals, typed joins, "find all *X* that *Y*-ed *Z*" — where a vector database struggles. The model is uniform: every fact is a triple of `<subject, predicate, object>`, where subjects and predicates are IRIs and objects are IRIs or literals. Turtle is the human-readable serialization; you read and write it almost always. Apache Jena Fuseki, running in Docker, is the triple store you will stand up locally and load. SPARQL is the query language; you will use `SELECT` for rows, `CONSTRUCT` for graphs, `ASK` for booleans, with `WHERE`, `FILTER`, `OPTIONAL`, `ORDER BY`, and `LIMIT` doing the heavy lifting. RDFS, OWL, and SKOS are the standard vocabularies — `rdfs:label` for human-readable names, `rdfs:subClassOf` for typing hierarchies, `owl:sameAs` for identity assertions across sources, and especially `skos:prefLabel` / `skos:altLabel` for resolving surface-form variants to a single canonical URI. From Python you use `rdflib` for in-memory work (the drill, validating local files) and `SPARQLWrapper` for talking to Fuseki (the lab and integration). Knowledge graphs and vector retrieval are complementary, not competing — Week B integrates them into a single semantic search pipeline.

What's Next

Sunday evening you do the **Core Skills Drill** — five SPARQL queries against a small fixture KG, run with `rdflib` in memory. It is the retrieval-under-pressure move that converts recognition into recall.

Monday you do the **Applied Lab** — extend a recipes ontology TTL, stand up Fuseki, load the dataset, and verify with three SPARQL queries (one of them uses SKOS to handle a label variant).

Thursday you do the **Integration Task** — an eight-query suite against a richer publications graph, exercising every SPARQL form and clause you have read about.

Going further (optional)

If you want to apply your Week A skills to the world's largest open knowledge graph, start with the Wikidata Query Service tutorial. The query patterns are the ones you just read about; only the dataset changes.

Looking ahead to Week B

Module 9 has two weeks because the field has two major production stacks for graph-shaped data. Week A is the **standards-grounded** vehicle — RDF / SPARQL / Fuseki — the substrate behind Wikidata, biomedical knowledge graphs, government and FAIR-data systems. Week B introduces a **production property-graph** stack — Neo4j — the substrate behind most 2026 LLM-application backends and modern recommendation / fraud / social-graph products. Both are real; the course teaches both. The concepts you build this week — pattern matching, identity, hierarchical traversal — carry directly into Week B; only the notation changes, and Week B's reading is where that mapping is taught.

© 2026 LevelUp Economy. All rights reserved. Unauthorized reproduction or distribution of this material is prohibited.